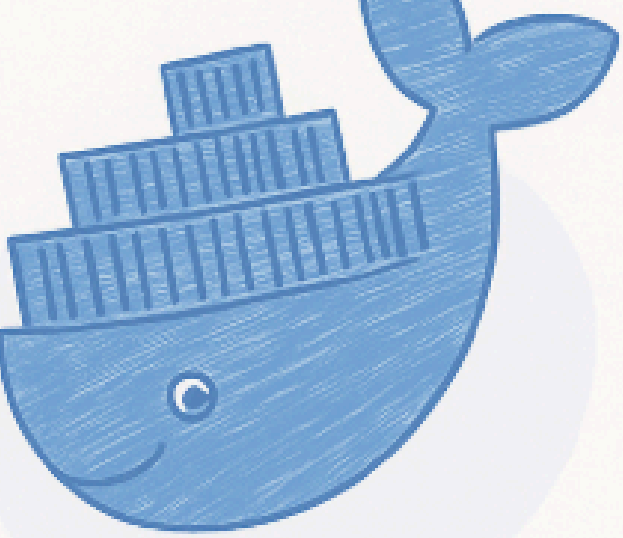
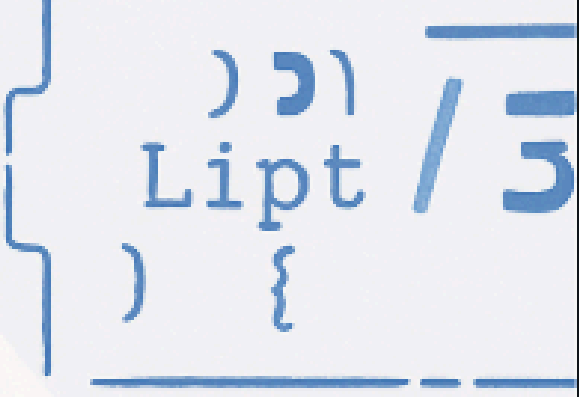




SAE 501

Projet de salles de TP virtuelles à l'aide de conteneurs

Jendryka Clément – Noa Etourneau – Hugo Defrance

1. Objectif général de la SAE..... 3
2. Choix de l'architecture et des technologies..... 3
3. Mise en place du portail web..... 4
4. Conception des environnements Linux (Prog, Réseaux, Cyber)..... 6
5. Intégration de Windows Server dans Docker..... 6
6. Gestion des conteneurs personnalisés..... 8
7. Sécurisation et contraintes du projet..... 8
8. Problèmes rencontrés et solutions apportées..... 8
9. Résultat final..... 9
10. On irait même plus loin ?..... 9

1. Objectif général de la SAE

L'objectif principal de cette SAE était de concevoir une plateforme centralisée permettant aux étudiants de lancer différents environnements de travaux pratiques : *un conteneur programmation (Linux graphique) / un conteneur réseaux (Wireshark) / un conteneur cybersécurité (EtherApe, nmap, etc.) / une machine virtuelle Windows Server intégrée dans Docker (Dockur)*

Le tout devait être accessible via un portail web, exécuté dans des conteneurs Docker, administrable via une interface simple, sécurisé (authentification, isolation), extensible (conteneurs personnalisés avec noVNC), fonctionnel sur une seule VM Ubuntu.

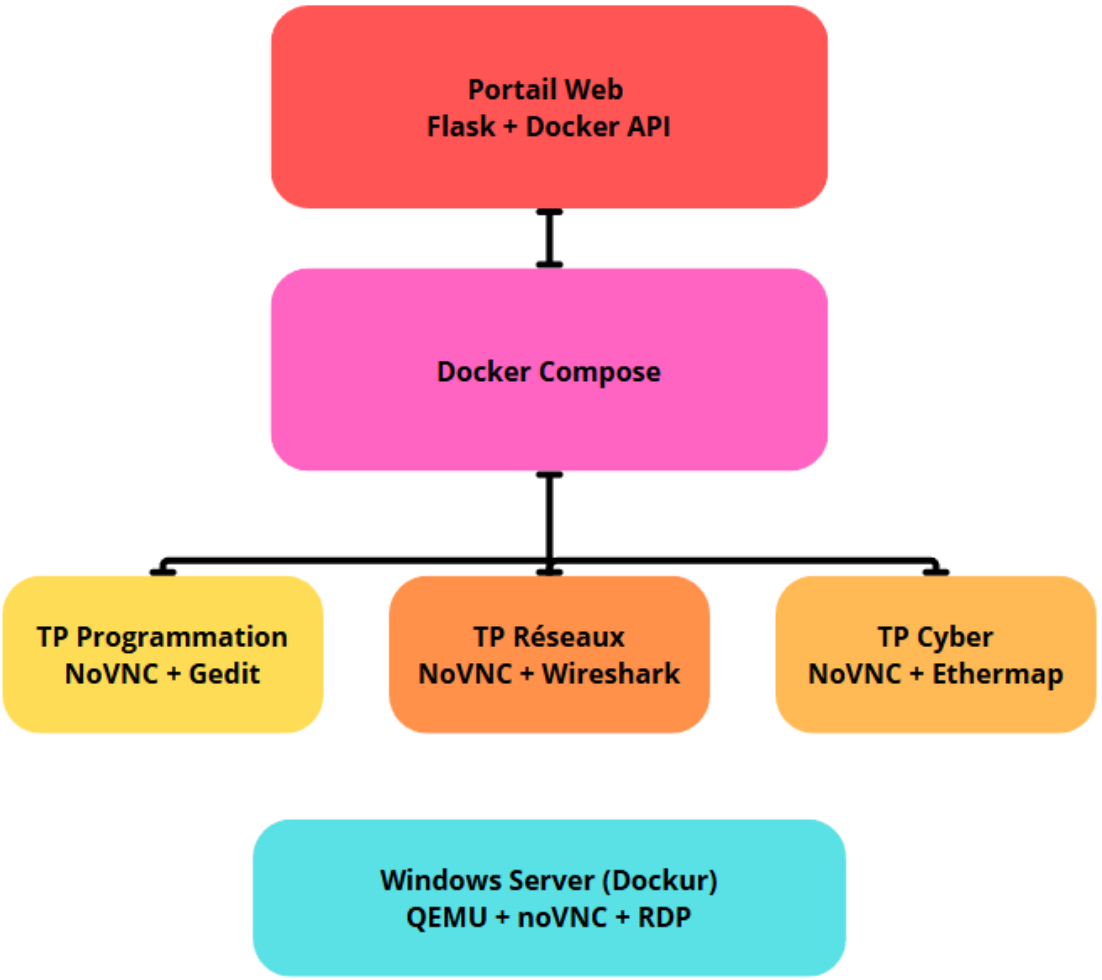
2. Choix de l'architecture et des technologies

Dès le début du projet, nous nous sommes orientés vers Docker, car il permet d'isoler proprement plusieurs environnements tout en restant léger et simple à déployer. Nous avons étudié d'autres solutions comme Kubernetes ou Docker Swarm, mais elles ont été écartées pour plusieurs raisons.

Kubernetes, par exemple, est conçu pour gérer des clusters distribués et assurer de la haute disponibilité. Il nécessite une architecture complexe (API Server, kubelet, scheduler, CNI…ces choses n'ont pas forcément étaient vu en cours) et consomme énormément de ressources. Dans notre cas, tout devait fonctionner sur une seule VM. Kubernetes aurait été trop lourd, trop complexe à gérer et inadapté aux contraintes.

Docker Swarm a également été écarté, principalement parce que la technologie n'est plus réellement maintenue par Docker et qu'elle n'apporte pas d'avantages concrets pour une architecture simple. Elle reste utile pour du multi-nœud, mais imposer Swarm pour une SAE de ce type aurait ajouté de la complexité inutile.

Docker Compose a donc été retenu, car il permet de définir toute la plateforme dans un simple fichier YAML, lisible et facilement maintenable. C'est un outil parfaitement adapté pour orchestrer quelques conteneurs sur une seule machine tout en gardant un niveau de contrôle élevé.



3. Mise en place du portail web

Nous avons développé un portail web en Flask qui sert de point d'entrée unique pour les utilisateurs. Ce portail permet :

-de se connecter (avec stockage des mots de passe hachés pour la sécurité)
-de lancer les environnements de TP
-de créer et supprimer des conteneurs personnalisés (réservé aux administrateurs)
-d'accéder directement au bureau graphique des conteneurs via noVNC

Ce portail a également permis de gérer proprement les droits : les étudiants peuvent simplement accéder aux TP, tandis que seuls les administrateurs ont la possibilité de manipuler des conteneurs personnalisés. Le fichier users.json contient les comptes, les rôles et les hash des mots de passe, ce qui évite toute vulnérabilité liée au stockage de données sensibles.

Exemple : Ce code montre comment Flask interagit avec Docker pour lancer un conteneur et rediriger l'utilisateur vers son bureau noVNC

```
@app.route("/tp/programming/start", methods=["POST"])
@login_required
def start_programming():
    host_port = start_env("prog-desktop:latest", "tp-prog-desktop", 26080)
    host = request.host.split(":")[0]
    return redirect(f"http://{host}:{host_port}/")
```

Commandes nécessaires pour lancer toute la plateforme :

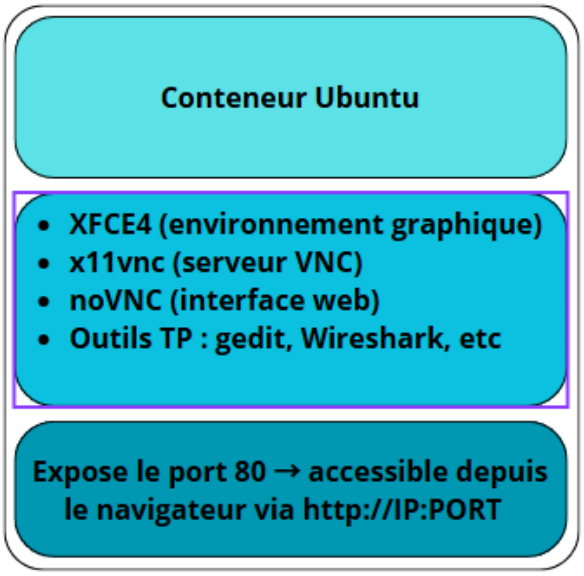
```
cd /home/administrateur/docker-manager

export http_proxy=http://cache-etu.univ-artois.fr:3128
export https_proxy=http://cache-etu.univ-artois.fr:3128
export HTTP_PROXY=$http_proxy
export HTTPS_PROXY=$https_proxy
export NO_PROXY="localhost,127.0.0.1,::1"
export no_proxy="localhost,127.0.0.1,::1"

docker compose up -d

docker compose ps
```

Portail de la plateforme : `http://172.31.28.155/`

4. Conception des environnements Linux (Prog, Réseaux, Cyber)

Nous avons créé trois images Docker basées sur Ubuntu et XFCE, avec accès graphique via noVNC.Chaque environnement embarque les outils correspondant au TP : programmation (éditeur gedit, Python…), réseaux (Wireshark, outils de diagnostic réseau), cybersécurité (EtherApe, utilitaires d'analyse)

noVNC a été choisi car il est extrêmement simple : il fonctionne directement dans le navigateur sans installation et permet une expérience utilisateur fluide.



5. Intégration de Windows Server dans Docker

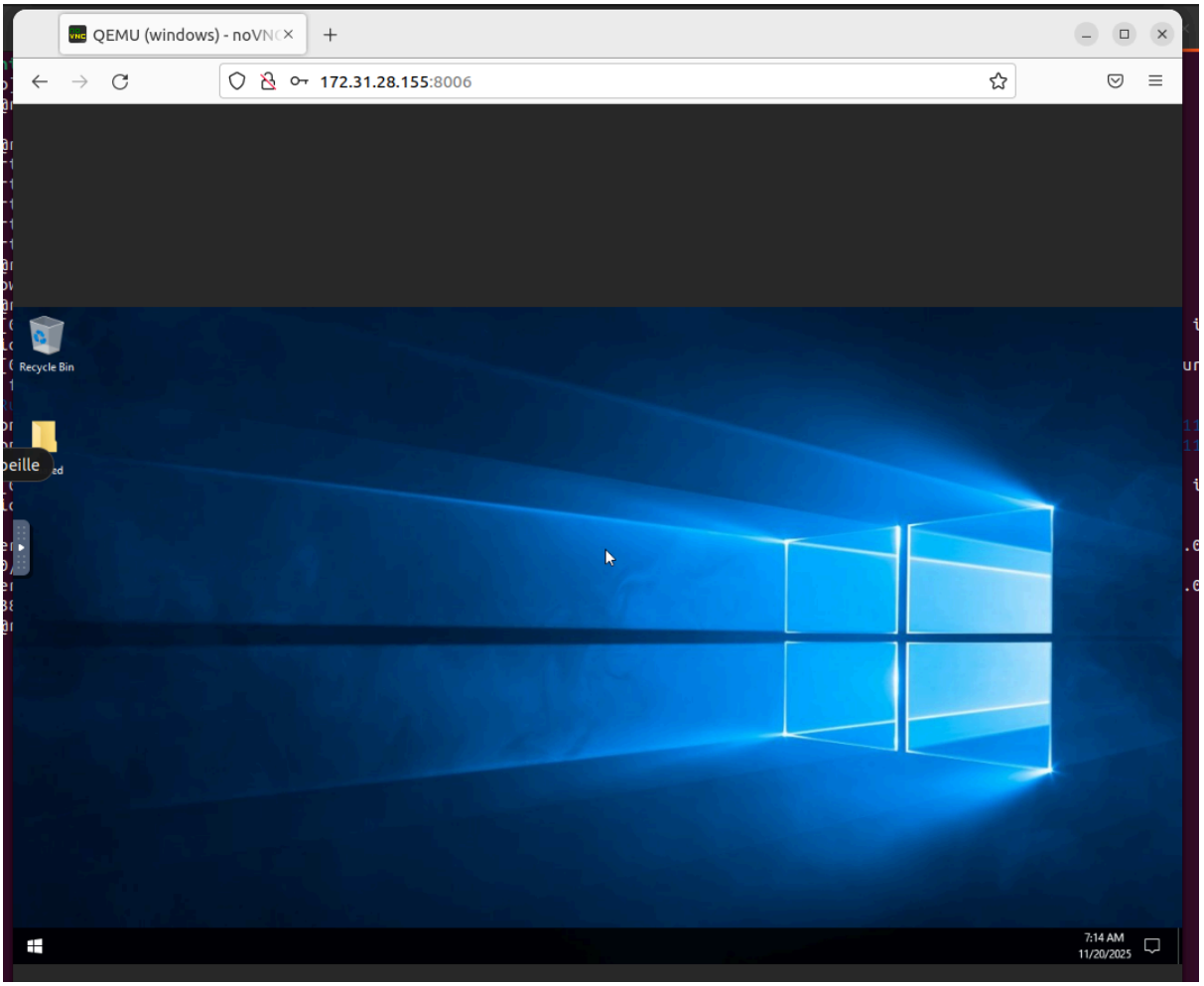
C'est la partie la plus complexe du projet. Docker ne peut pas exécuter Windows Server nativement dans un conteneur Linux. Nous avons donc utilisé la solution Dockur/Windows, qui exécute Windows à l'intérieur d'un conteneur via QEMU. Cela nous a permis d'obtenir un Windows Server opérationnel, accessible en noVNC et en RDP. Plusieurs difficultés ont été rencontrées :

Le proxy de l'IUT bloquait le téléchargement de l'ISO Windows, ce qui provoquait une boucle infinie. Nous avons dû configurer toutes les variables http_proxy et https_proxy dans Docker.
L'absence de KVM dans VirtualBox provoquait une erreur QEMU, empêchant le lancement. La solution a été de désactiver l'accélération matérielle (`KVM=N`).
La VM Ubuntu n'avait pas assez d'espace disque pour stocker Windows, ce qui causait des erreurs "No space left on device". Nous avons augmenté la taille du disque virtuel puis étendu la partition.
Windows Server étant volumineux, le premier lancement prend du temps à cause du téléchargement et de l'extraction.

Après résolution de ces points, Windows Server fonctionnait parfaitement depuis le navigateur.

```
winserver:
  image: dockurr/windows
  container_name: docker-manager-winserver
  environment:
    - VERSION=2016
    - KVM=N
    - RAM_SIZE=3G
    - http_proxy=http://cache-etu.univ-artois.fr:3128
    - https_proxy=http://cache-etu.univ-artois.fr:3128
  volumes:
    - ./windows:/windows
  ports:
    - "8006:8006"
    - "3389:3389"
```

Ce service permet de télécharger, démarrer et accéder à Windows Server directement depuis le navigateur.



6. Gestion des conteneurs personnalisés

L'une des fonctionnalités importantes du portail est la possibilité, pour l'administrateur, de créer des conteneurs personnalisés basés sur des images existantes (prog, réseau, cyber). Le portail attribue automatiquement un port disponible, démarre le conteneur via l'API Docker, et l'affiche dans l'interface. Chaque conteneur peut être supprimé proprement.

Cette fonctionnalité permet d'étendre la plateforme en fonction des besoins des TP ou de tests spécifiques.

7. Sécurisation et contraintes du projet

La sécurité repose principalement sur l'isolation naturelle des conteneurs Docker, les mots de passe hachés, la séparation des rôles (admin / user), l'absence de persistance entre utilisateurs, l'accès strict via le portail

Nous avions commencé à intégrer le HTTPS via Nginx et des certificats auto-signés, mais cette partie a été abandonnée pour ne pas introduire de risques à la veille de la soutenance. Le navigateur avait mémorisé que l'adresse devait être accédée en HTTPS, ce qui provoquait des erreurs même après suppression du proxy (nous avons donc retiré le cache de Firefox puis nous pouvons de-nouveau y accéder en HTTP). Pour garantir à 100% la stabilité, nous avons gardé une version simple en HTTP interne. (Manque de temps)

8. Problèmes rencontrés et solutions apportées

Durant la conception, nous avons rencontré plusieurs problèmes techniques significatifs :

-Les images Docker ne se téléchargeaient plus en build : même problème de proxy, résolu avec des variables d'environnement.
-Windows Server refusait de démarrer : désactivation de KVM et augmentation du stockage.
-Le navigateur forçait le HTTPS : navigation privée et suppression des réglages, puis abandon du HTTPS.
-Certains conteneurs restaient en "Restarting" : erreurs liées à l'espace disque ou au proxy.
-Wireshark demandait des droits root : nous avons installé la version graphique dans les conteneurs.

Ces problèmes nous ont obligés à comprendre comment les régler et donc par conséquent prendre du retard.

9. Résultat final

La plateforme obtenue fonctionne parfaitement :Le portail web est clair, moderne et simple à utiliser, les utilisateurs peuvent se connecter et accéder aux TP immédiatement, les environnements programmation, réseau et cybersécurité fonctionnent sans installation locale, Windows Server est opérationnel via le navigateur, l'administrateur peut créer et supprimer des conteneurs personnalisés, l'architecture est solide, cohérente et adaptée aux contraintes du projet.

Cette SAE nous a permis d'aborder des notions avancées : conteneurisation, virtualisation, orchestrateurs, interactions Docker–Flask, sécurité web, QEMU, noVNC et administration système.

10. On irait même plus loin ?

Nous avions également prévu d'ajouter une fonctionnalité importante que nous n'avons pas pu finir par manque de temps : la séparation complète des bureaux noVNC entre les utilisateurs. L'idée était que chaque étudiant puisse avoir un environnement totalement personnel. Par exemple, si le "user1" créait un fichier appelé *test*, le "user2" ne devait pas le voir lorsqu'il ouvrait son propre environnement.

Pour permettre cela, il aurait fallu que la plateforme crée automatiquement un conteneur différent pour chaque utilisateur, au lieu d'utiliser le même pour tout le monde. Concrètement, nous aurions dû lancer des conteneurs avec un nom unique par utilisateur (comme `tp-prog-user1`, `tp-prog-user2`, etc.) et leur attribuer chacun un port distinct. Cela aurait permis de garantir que les fichiers, les paramètres et les manipulations restent privés pour chaque compte.

Même si cette fonctionnalité n'a pas été réalisée, elle reste une évolution logique et assez simple à ajouter plus tard. Pour la soutenance, nous avons préféré rester sur une version stable et fiable, plutôt que de risquer d'introduire des bugs à la dernière minute.

Une autre amélioration que nous aurions aimé intégrer concerne la sécurisation avancée de la plateforme. Nous avons mis en place l'authentification et le hachage des mots de passe, mais nous voulions aller plus loin en ajoutant une gestion d'accès plus fine, une meilleure isolation réseau entre les conteneurs, ou même un accès chiffré via HTTPS. Nous avions d'ailleurs commencé à travailler sur une configuration Nginx avec des certificats SSL, mais cette partie s'est révélée trop instable à cause des contraintes du proxy de l'IUT et du navigateur qui forçait le HTTPS. Par manque de temps, nous avons préféré rester sur une version simple et fiable en HTTP, mais l'intégration d'un vrai HTTPS serait une évolution naturelle.

Nous aurions également pu mettre en place un monitoring des conteneurs pour suivre leur consommation en ressources, détecter les pannes ou permettre à l'administrateur de superviser l'activité de la plateforme. Des outils comme Portainer ou Grafana auraient pu être intégrés, mais cela aurait demandé davantage de configuration et aurait compliqué l'architecture générale.

Enfin, nous avons envisagé la possibilité de sauvegarder automatiquement certains conteneurs pour permettre à un étudiant de retrouver son environnement exactement comme il l'avait laissé, ou même d'exporter ses travaux. Cette fonctionnalité demande une gestion avancée des volumes et aurait nécessité de reconstruire une partie de la plateforme.

Ces améliorations n'ont pas été réalisées principalement par manque de temps et parce que notre priorité était d'obtenir une plateforme opérationnelle, stable et simple pour la soutenance. Elles constituent néanmoins des pistes réalistes pour faire évoluer le projet dans le futur.